# GNR650: Advanced Topics in Deep Learning for Image Analysis

(Autumn 2026-27)

**Programming Assignment:** Self Supervised and Meta Learning

---

## 1 Introduction

In this assignment, you will investigate and build deep learning architectures that build robust representations and evaluate them using linear probing on a noisy labelled dataset. You will first leverage self-supervision learning (SSL) paradigm (using SimCLR, DINO, MAE or JEPA) of your choice to build robust encoding without annotations. You will then add a linear classifier while keeping the SSL encoder fixed on the provided labelled dataset where some of the training labels are corrupted. During testing, you'll also encounter classes that were not present in the training data and your model must correctly classify it as "Unknown" class.

You may use Pytorch, TensorFlow or JAX but any high level model repository (e.g. torchvision models, timm, hugging face, etc.) and specialised Out of distribution and Self-supervised learning libraries are strictly prohibited.

You must use only the dataset provided for this assignment. The use of any additional datasets, external training data, pretrained models, or pretrained weights is **strictly prohibited**. Your final model will be evaluated on a held-out test dataset. The test dataset must not be used during training, hyperparameter selection, architecture selection, or any other form of model development.

In addition to achieving good classification accuracy, your implementation must finish the complete test-set inference within 1 hour on the evaluation system. The final evaluation will consider both classification accuracy and computational efficiency.

**Submission Requirements:**

You must submit:

1. Technical report describing your implementation, experiments, results, and analysis on Moodle.
2. Complete source code, pushed to your private github repository.
3. Trained model checkpoints are required to reproduce your final results.
4. Any required configuration, setup, environment files, and scripts needed to run your implementation.

Your repository must contain a working setup script and test script. The following commands will be used to verify your submission:

Test Script

```
bash setup.bash
python test.py --test-csv <absolute-path-to-csv-containing-file-path> --output-path <
    absolute-path-to-store-output-csv-file>
python grade.py --output-path <absolute-path-to-store-output-csv-file>
```

Your code must execute successfully using the above procedure without requiring manual modification of the source code. The evaluation will use the submitted implementation and checkpoints to independently verify the reported accuracy, inference time, and other required performance metrics.

**Please use your IITB email ID to access any of the links provided.**

## 2 Learning Outcomes

Upon successful completion of this assignment, students will be able to:

- Implement Self-Supervised Learning (SSL) algorithms from scratch.
- Linear Probing under noisy labelled dataset to train a robust model.
- Develop Out of distribution (OOD) detection techniques to identify novel, unseen class during inference.

# 3 Assignment Requirements

The assignment consists of three core components. All students must complete the core components.

## 3.1 Core Component 1 (Self-Supervised Feature Learning)

Implement a self-supervised representation framework from scratch on the provided unlabeled dataset. You may select one of the following self-supervised paradigm (these are suggestion, you are free to explore other algorithms as well):

- Contrastive Learning Framework like SimCLR
- Self-Distillation Framework like Dino
- Masked Image Modelling Framework like MAE
- Joint Embedding Predictive Architecture like JEPA

The complete backbone and feature encoder must be trained from random initialization without relying on pretrained weights or pre-built model libraries.

## 3.2 Core Component 2 (Linear Probing on Noisy Labels)

In this component, you will evaluate the robustness of your pre-trained SSL representation under corrupted and limited labeled data scenario. You will execute the following:

- **Linear Classifier Setup:** Attach a single linear classification layer on top of the frozen SSL feature encoder
- **Noisy Training Set:** Train the linear layer on provided dataset where the class labels are corrupted.

You should analyze the results against a baseline supervised model (same backbone + linear head) trained end-to-end from scratch directly on the noisy data by keeping all experimental setup same.

## 3.3 Core Component 3 (Unknown Class Detection)

During testing, your model will be evaluated on a hidden evaluation dataset containing both known classes (seen during training) and novel, unseen classes. Implement the method to handle unseen classes and assign it to label K (representing the (K+1)-th class where known class, seen during training, are indexed $0, 1, 2, ..., K-1$ )

## 3.4 Dataset Loading

You have to download the dataset from the folder Assignments 2 from the provided link `https://drive.google.com/drive/folders/1UOuBLtVzOzt-wGPFAsqifmn0fiGlC7bz?usp=sharing`. The dataset provided consists of two main subdirectories: `unlabelled_data` and `labelled_data`. You must use `unlabelled_data` to train your Self-supervised model and `labelled_data` to train the linear classifier.

The `labelled_data` containes noisy labels and will be in image-folder format, i.e., parent folder consists of multiple subdirectories, each containing images in PNG format. The exact dataset directory layout is structured as follows:

```
dataset/
├── unlabelled_data
│   ├── img_0001.png
│   ├── img_0002.png
│   └── ...
└── labelled_data
    ├── class_0
    │   ├── img_0021.png
    │   ├── img_0031.png
    │   └── ...
    └── class_1
        └── img_1021.png
```

```
    ├── img_1031.png
    ├── ...
├── ...
└── class_K-1
    ├── img_0121.png
    ├── img_0231.png
    └── ...
```

### 3.5 Constraints

- The complete inference on the hidden test dataset must finish within 1 hour on the evaluation machine.
- Pre-trained models, pre-trained weights, and externally trained checkpoints are strictly prohibited.
- Only the provided dataset may be used for training and experimentation.
- Students must not use additional external datasets.

## 4 Languages and Tools

- **Target OS:** Linux
- **Language:** C++ or Python (either may be used)
- **CUDA Version:** 13.x

## 5 Evaluation & Grading Strategy

### 5.1 Evaluation

Submission will be evaluated based on the performance on the evaluation leaderboard. A detailed breakdown is provided in Table 1. You must push your code to the GitHub repository link submitted in the form along with the report on **moodle**. The repository must contain, at minimum, a setup script and a test script.

| S.No. | Assignment Component | Weight (%) |
|---|---|---|
| **1** | Leaderboard | 50 |
| **2** | Report | 10 |
| **3** | Beating the provided benchmark | 5 |
| **4** | Listing down all the sources used for assignment | 5 |
| | **Total** | **70** |

Table 1: Grading Breakdown

During automated evaluation, a hidden evaluation dataset will be provided. The absolute path to a hidden test CSV file and an output directory path will be supplied to your test script. The evaluation environment will execute the following commands:

Test Script

```
bash setup.bash
python test.py --test-csv <absolute-path-to-csv-containing-file-path> --output-path <
    absolute-path-to-store-output-csv-file>
python grade.py --output-path <absolute-path-to-store-output-csv-file>
```

The input CSV file provided contains the list of hidden test instances formatted as follows:

Input Test CSV (test.csv)

```
id,file_path
```

```
0,/path/to/test_images/test_00001.png
1,/path/to/test_images/test_00002.png
2,/path/to/test_images/test_00003.png
```

Your `test.py` script must process the test images, load your submitted model checkpoints, perform inference and save a CSV file named **predictions.csv** in the provided director `--output-path`. The generated `predictions.csv` file **must strictly follow this structure**:

Generated Output CSV (predictions.csv)

```
id,file_path,label
0,/path/to/test_images/test_00001.png,0
1,/path/to/test_images/test_00002.png,12
2,/path/to/test_images/test_00003.png,K
```

Where:

- label $\in \{0, 1, 2, \ldots, K-1\}$ represents the predicted class for samples belonging to one of the $K$ known training classes.
- label $= K$ represents samples detected as out-of-distribution or novel unseen classes (**"Unknown"** class).

**READ VERY CAREFULLY:**

- You will be provided absolute path to csv file including the file name for `--test-csv`
- You will be provided absolute directory path to output directory **without file name** and you should create `predictions.csv` yourself.
- The CUDA version of the target machine will be 13.x
- If the submitted code fails to run using the provided instructions, cannot load the supplied test dataset, or requires modifications to the submitted source code in order to perform the evaluation, the submission will receive a grade of 0.

In addition to the code, you must submit a written report that clearly documents:

- Model architecture and the rationale behind the architectural choices
- Total number of trainable parameters
- MACs and FLOPs of your final model
- Experimental results
- Any additional key indicators relevant to the implemented architecture and assignment components
- At least one failed design decision or experiment, including an explanation of why it did not work and what was learned from it

A baseline benchmark will be provided. The benchmark will specify the minimum expected performance under the evaluation protocol. Failure to report the required metrics or significant omissions in the documentation may result in loss of marks. You must cite all resources used while completing this project, including papers, documentation, tutorials, code repositories, and AI tools. You must also follow the honour code specified in the course.